



S6/b Consensus Testing Summary

Verification Evidence for the S6/b Testing-Mode Implementation

Rincoin Community Forge

2026-08-25

Table of contents

1 Purpose of This Document	2
2 What Changes at H1, Briefly	2
3 A Design Decision, Reviewed Before Implementing	3
4 Summary	4
5 Methods: What Was Tested, and How	4
5.1 Unit-level	5
5.2 Single-node functional	5
5.3 Multi-node functional	5
5.4 Cross-implementation	6
6 Results: Did It Pass?	6
6.1 Unit-level — pass	6
6.2 Single-node functional — pass	7
6.3 Multi-node functional — pass	7
6.4 Cross-implementation — partially done, passing so far	7
7 Scope: What This Testing Does Not Cover	7

 Discussion Draft
Document version 0.1 · 2026-08-25

No scenario has been selected, preferred, or adopted. This document reports testing evidence for a **testing-mode-only** implementation of the S6/b candidate; it does not argue for S6/b over S1 or S5/b, and it is not a readiness or deployment claim.

 Editorial note — for maintainers, not part of the reported evidence

This branch is pushed to `rincoin-community/rincoin-core`; CI has concluded green on the commit below (see Provenance), confirming the results reported here rather than resting on local verification alone. The cross-implementation producer × validator matrix and two narrow commitment test-vector cases (inherited from S1, not new here) are also outstanding. The full, current list is [S6B/open-items.md](#).

1 Purpose of This Document

This is a management summary, not a specification. It reports whether the S6/b testing-mode implementation actually behaves as `technology/consensus-transition.md` and `analysis/Rincoin_840k_S6B_Consensus_Change_Specification.qmd` say it should — checked by running it, not by re-reading the design. The full evidence — every test case, raw logs, exact commits — is in [S6B/](#), not repeated here.

This scenario’s testing-mode branch was built by branching directly from the already-tested `consensus/s1-testing` branch and reusing its shared infrastructure (coinbase commitment, `sig_fork_id`, mainnet blocker, wallet/PSBT signing) unmodified. Unlike S5/b, S6/b’s schedule needed a genuine new design decision, not just a new formula — see “A Design Decision, Reviewed Before Implementing” below.

2 What Changes at H1, Briefly

At block height 840,000 (mainnet), three things change together: the block subsidy schedule, a scheduled coinbase commitment identifying the chosen schedule, and a sighash-level replay identifier (`sig_fork_id`) mixed into new transaction signatures. The commitment and `sig_fork_id` mechanisms are identical across every candidate scenario; only the subsidy schedule differs. S6/b’s schedule: four fixed-value phases (4, 2, 1, 0.6 RIN) at heights 840,000 / 2,100,000 / 4,200,000 / 6,300,000, followed by a hard cutoff to exactly zero at height 234,587,500 — a height *derived* from an exact 168,000,000 RIN issuance ceiling, not a designed round number. Full rationale and byte-level specification: `analysis/Rincoin_840k_S6B_Consensus_Change_Specification.qmd`. This document assumes that design and tests it; it does not re-argue it.

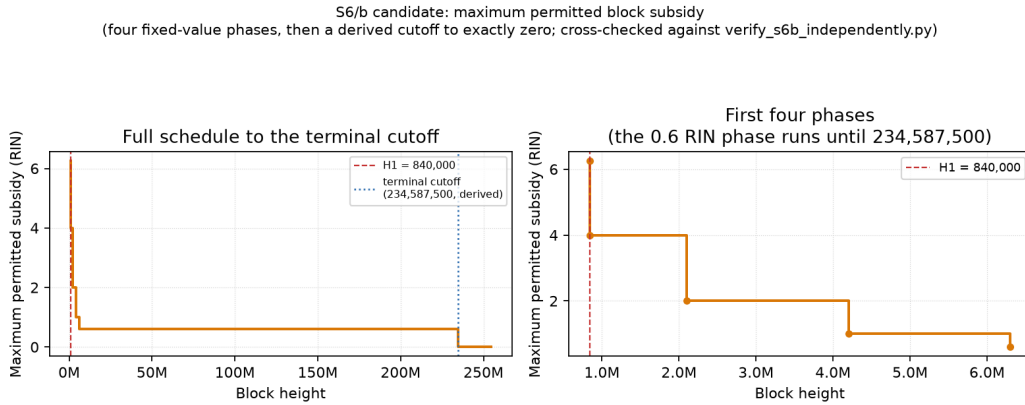


Figure 1: Maximum permitted block subsidy under the S6/b candidate. Unlike S1/S5/b, this schedule has no exponential decay to sample densely – it is exactly five flat segments, transcribed directly from the specification and cross-checked against `analysis/data/S6B_normative_test_vectors.csv` and `analysis/scripts/verify_s6b_independently.py`.

3 A Design Decision, Reviewed Before Implementing

S1’s and S5/b’s subsidy formulas are closed-form (a recursive rule, a binary-shift rule) — the C++ function that computes them is a few lines of arithmetic. S6/b has no closed form: it’s four fixed values over four height ranges, then a cutoff. Before writing that function, and at the user’s explicit request, a structurally similar existing implementation was reviewed for lessons: `rincoin-core/rincoin@v1.1`’s own “Customized Halving” subsidy code (a different, independently developed fork — one associated with a rival replay-protection proposal `technology/response-to-rip-0009.md` already responds to, not a neutral reference).

What was good, and adopted: that code expresses its phase boundaries as multiples of `nSubsidyHalvingInterval` rather than hardcoded mainnet heights, so mainnet and fast test networks share one code path with proportionally scaled timing. This directly solves the “how do you make a 228-million-block terminal phase reachable in a regtest functional test” problem this scenario has to solve. This branch’s own `ForkSubsidyPhases` entries are expressed relative to `ForkH1Height` for the same reason.

What was not adopted: that code is a hardcoded, descending `if/else if` chain with bare magic numbers, and — the more important finding — it has **no terminal cutoff at all**. Its last phase pays 0.6 RIN forever, confirmed from the very first commit that introduced it, making its total issuance unbounded. S6/b’s own specification derives an exact terminal height specifically so issuance is bounded to exactly 168,000,000 RIN. Both call themselves “Scenario II”; they are not the same rule. Adopting that code’s shape as-is would have silently implemented a different, unbounded scenario.

What was built instead: `Consensus::Params::ForkSubsidyPhases`, a plain, ordered, variable-length table of `{offset_from_h1, subsidy}` entries, populated per network in `chainparams.cpp`. `GetBlockSubsidyPostFork()` is a generic scan over this table — nothing in it is specific to having exactly four phases. Extending or changing the schedule later is a data change to the table, not a rewrite of validation logic.

4 Summary

Every test that has been run passes. 532 of 532 C++ unit tests, 159 of 159 functional tests, and every cross-implementation comparison run so far — on the first attempt, with no fix cycle needed (unlike S5/b, whose test suite needed a real fix for a structural MWEB-height interaction).

Evidence layer	Result	Status
C++ unit tests	532 / 532	Done
Fork scenario functional tests (single- and two-node)	6 / 6	Done
Full functional CI allowlist (fork + pre-existing, unmodified)	159 / 159	Done
Independent terminal-height/ceiling verification (pre-existing script, reused)	exact match	Done
Third-party implementation design review (rincoin-core/rincoin@v1.1)	reviewed, not adopted as-is	Done
Cross-implementation producer x validator matrix	not yet run	Outstanding
Fuzz targets (commitment parsing, sig_fork_id sighash mixing)	not yet written	Outstanding
Pool / seeder integration rehearsal	not yet run	Outstanding
Push to rincoin-community/rincoin-core + CI run	not yet done	Outstanding

Two evidence layers are not yet complete, both inherited from S1 rather than newly discovered here — see Results §5.4 and [S6B/open-items.md](#).

5 Methods: What Was Tested, and How

Four groups, used consistently through both this section and Results — the same four used for S1 and S5/b:

Group	What it checks	Why it's a separate group
Unit-level	Each piece of consensus logic against frozen, byte-exact vectors	Smallest testable unit; catches an error before it can reach anything that depends on it
Single-node functional	The same rules, enforced end-to-end by a real running node via RPC/P2P	Confirms the rules hold when driven the way a real deployment would drive them
Multi-node functional	Whether the network prefers higher work only when that chain is also rule-valid	The core guarantee the coinbase-commitment mechanism exists to provide
Cross-implementation	What happens when this build meets software that disagrees with it	The only layer that reflects the network as it actually is

5.1 Unit-level

The subsidy phase table is checked against `analysis/data/S6B_normative_test_vectors.csv` (every phase boundary and the terminal cutoff) and independently against `analysis/scripts/verify_s6b_independently.py`, a pre-existing script that derives the terminal height from the issuance ceiling without importing the C++ implementation — reused directly rather than duplicated, since it already does exactly what S1's and S5/b's own from-scratch re-derivation scripts do for their scenarios. The commitment builder/parser and `sig_fork_id` sighash-mixing code are reused unmodified from `consensus/s1-testing`.

5.2 Single-node functional

Driven through a real, running `rincoind` process via its actual RPC and P2P interfaces: coinbase-commitment validation (reused unmodified from S1), and subsidy enforcement across all five phases of the regtest table, including the terminal zero phase — see Results §5.2 for why this scenario could test more of its own schedule directly than S1 or S5/b could.

5.3 Multi-node functional

Reused unmodified from S1: two testing-mode nodes, one isolated to mine a commitment-valid and a commitment-invalid competing chain; which one is adopted is the test.

5.4 Cross-implementation

Every accessible node binary tested as both block *producer* and block *validator* against every other binary.

i Editorial note — for maintainers

Same outstanding item as S1 and S5/b: `test/fork_interop_matrix.py` is not yet built. The two narrower, hand-written comparisons stand in for it.

6 Results: Did It Pass?

6.1 Unit-level — pass

532 of 532 C++ unit tests pass. The subsidy phase table matches the specification's own normative vectors at every boundary and at the derived terminal cutoff:

Height	Vector ID	CSV value (base units)	C++ unit test result	Note
839,999	S6B-001	625,000,000	625,000,000 — match	last pre-activation block
840,000	S6B-002	400,000,000	400,000,000 — match	activation block
2,100,000	S6B-005	200,000,000	200,000,000 — match	first 2 RIN block
4,200,000	S6B-008	100,000,000	100,000,000 — match	first 1 RIN block
6,300,000	S6B-011	60,000,000	60,000,000 — match	first 0.6 RIN block
234,587,499	S6B-014	60,000,000	60,000,000 — match	final non-zero block
234,587,500	S6B-015	0	0 — match	first zero-subsidy block

The pre-existing independent verification script confirms the terminal height and issuance ceiling exactly:

Independent S6/b verification passed: first zero height 234,587,500; exact maximum 168,000,000

The commitment-parsing vector catalog carries the same two open exceptions S1's and S5/b's own annexes track (not new to this scenario): see [S6B/commitment-test-vectors.md](#).

Full detail: [S6B/unit-test-results.md](#).

6.2 Single-node functional — pass

All commitment-validation checks pass, reused unmodified from S1. Subsidy enforcement was tested across **all five** phase transitions in this scenario’s regtest table — including the terminal zero cutoff itself, not just one later boundary — because S6/b’s own small-scale table happens to stay well under the regtest MWEB activation height across its entire range, unlike S5/b’s second boundary. This is meaningfully stronger coverage than either S1’s or S5/b’s own subsidy tests, which each only exercise two data points.

6.3 Multi-node functional — pass

Both reorg outcomes are correct, reusing S1’s own mechanism test unmodified.

6.4 Cross-implementation — partially done, passing so far

The full producer × validator matrix has not been run. The two narrower comparisons that are done both pass, with the same conclusions already established for S1 and S5/b (this mechanism doesn’t depend on the subsidy scenario): clean divergence from v1.1.0 and v1.0.1, correct header/full-block asymmetry against the foreign Aevust/rincoin fork.

A fourth, non-automated comparison belongs in this section too: the rincoin-core/rincoin@v1.1 design review described above. It isn’t a pass/fail test, but it directly shaped what got built, and its finding (an existing, structurally similar implementation that’s actually unbounded) is exactly the kind of evidence a “does this withstand real conditions” report should carry.

Full detail: [S6B/functional-test-results.md](#).

7 Scope: What This Testing Does Not Cover

Same deliberate scoping as S1 and S5/b — not repeated in full here. See [S1’s own “Scope” section](#).

8 Provenance

Repository	rincoin-community/rincoin-core
Branch	consensus/s6b-testing
Branched from	consensus/s1-testing @ a8617fc73 (post branch_id migration)
Commit tested	1e5a4201d

CI run	github.com/rincoin-community/rincoin-core/actions/runs/33994483613 — concluded, success (all three jobs green; unit+functional’s first attempt hit a shared <code>feature_taproot.py</code> CI-runner flake, confirmed transient by a clean rerun)
branch_id	00112233445566778899aabbccddeeff — the specification’s own canonical synthetic test value, inherited directly
scenario_id	3 — provisional, ad-hoc, pending official upstream assignment (S1=1, S5/b=2, S6/b=3 by convention)
Build	testing-mode only; refuses to start on mainnet unless <code>RINCOIN_TESTING_ALLOW_MAINNET=1</code> is set in the process environment

9 Where to Go Next

- Full evidence, raw results, exact reproduction commands: [S6B/](#)
- The mechanism being tested: `technology/consensus-transition.md`, `technology/replay-protection-plan.md`
- The normative rule and its own vector set: `analysis/Rincoin_840k_S6B_Consensus_Change_Specif`
- Outstanding work tracked for maintainers: [S6B/open-items.md](#)